



GitHub — Complete Notes

Git hosting & collaboration · Code, review, automate, ship



A reader-friendly, all-in-one reference for GitHub.

Table of Contents

1. What is GitHub
2. Git vs GitHub
3. Core Concepts
4. Repositories
5. Essential Git Workflow
6. Branching & Merging
7. Pull Requests
8. Forks & Collaboration Models
9. Issues & Project Management
10. GitHub Actions (CI/CD)
11. Releases, Tags & Packages
12. Authentication & Security
13. GitHub CLI & Useful Commands
14. Best Practices
15. Quick Mental Model

1. 🐙 What is GitHub

GitHub is a **cloud platform for hosting Git repositories** and collaborating on code. On top of Git's version control it adds **pull requests, code review, issues, CI/CD (Actions), packages, releases, security scanning, and project management** — making it the hub where teams (and open source) build software together.

💡 **Mental model:** **Git** is the version-control engine (local). **GitHub** is the place your repos live online plus the collaboration + automation layer around them.

Common uses: source hosting, team collaboration & code review, open-source projects, CI/CD pipelines, issue tracking, documentation (wikis/Pages), and release distribution.

2. Git vs GitHub

	Git	GitHub
What	Distributed version-control tool	Hosting + collaboration platform built on Git
Where	Runs locally (CLI)	In the cloud (web + API)
Made by	Linus Torvalds (2005)	GitHub, Inc. (now Microsoft)
Works offline	 Yes	Needs network to sync
Adds	Commits, branches, history	PRs, issues, Actions, reviews, access control

Other Git hosts exist (GitLab, Bitbucket, Gitea) — they host the **same Git repos**; GitHub is the most widely used.

3. Core Concepts

Concept	Description
Repository (repo)	A project's files + full version history.
Commit	A snapshot of changes with a message + author.
Branch	A movable pointer / independent line of work.
Remote	A hosted copy of the repo (e.g. <code>origin</code> on GitHub).
Clone	A full local copy of a remote repo.
Fork	Your own server-side copy of someone else's repo.
Pull Request (PR)	A proposal to merge one branch into another, with review.
Issue	A tracked task, bug, or discussion.
Action / Workflow	Automated CI/CD jobs triggered by events.
Release / Tag	A named, versioned snapshot for distribution.

4. Repositories

- A repo holds your code, history, branches, issues, PRs, and settings.
- **Visibility:** **Public** (anyone can view), **Private** (you control access), or **Internal** (org-wide, Enterprise).

- Useful repo files: **README.md** (front page), **LICENSE**, **.gitignore**, **CONTRIBUTING.md**, **CODEOWNERS**, **.github/** (templates + workflows).
- **Branch protection rules** enforce review, passing checks, and linear history on important branches (e.g. **main**).
- Extras: **Wiki**, **GitHub Pages** (free static site hosting), **Discussions**, **Projects**.

```
gh repo create my-project --public --clone    # create + clone (GitHub CLI)
git clone https://github.com/user/repo.git    # clone an existing repo
```

5. Essential Git Workflow

The everyday loop — edit, stage, commit, push:

```
git status          # what changed
git add file.txt     # stage a file (git add -A = everything)
git commit -m "feat: add X" # snapshot staged changes
git push origin main # upload commits to GitHub
git pull origin main  # fetch + merge remote changes
git log --oneline -10 # recent history
git diff             # unstaged changes
```

The three areas: Working directory (your edits) → **git add** → Staging area (index) → **git commit** → Repository (history) → **git push** → GitHub.

💡 **git fetch** downloads remote changes **without** merging; **git pull** = **fetch** + **merge** (or **--rebase**).

6. Branching & Merging

- Branch off **main** for each feature/fix → isolate work, keep **main** releasable.
- Merge strategies on GitHub PRs:
 - **Merge commit** — preserves all commits + a merge node (full history).
 - **Squash and merge** — combines a PR into one tidy commit (clean history; common default).
 - **Rebase and merge** — replays commits linearly, no merge commit.
- **Merge conflicts** happen when two branches change the same lines — Git marks them; you resolve, then commit.

```
git switch -c feature/login    # create + switch to a branch (or: git checkout -b)
git switch main                # switch branches
git merge feature/login        # merge a branch into current
git rebase main                # replay current branch on top of main
git branch -d feature/login    # delete a merged branch
```

⚠️ **Rebase rewrites history** — never rebase commits already pushed and shared, or you'll diverge from collaborators.

7. Pull Requests

A **PR** proposes merging a branch and is the heart of GitHub collaboration:

1. Push a feature branch and **open a PR** against the base (e.g. `main`).
 2. **Reviewers** comment, request changes, or approve; CI checks run automatically.
 3. Address feedback with more commits; when approved + green, **merge**.
- **Draft PRs** signal work-in-progress (no review yet).
 - **Required reviews + status checks** (branch protection) gate the merge.
 - Reference issues in the description to auto-close them: `Fixes #123`.
 - **CODEOWNERS** auto-requests the right reviewers for touched paths.

```
gh pr create --title "feat: login" --body "Fixes #123"
gh pr status          # PRs involving you
gh pr checkout 42     # check out PR #42 locally
gh pr merge 42 --squash --delete-branch
```

8. Forks & Collaboration Models

Model	How	Best for
Shared repo	Collaborators push branches to one repo, open PRs	Teams / orgs with write access
Fork & PR	Fork → push to your fork → PR to upstream	Open source / external contributors

- A **fork** is your own copy; keep it in sync with **upstream**:

```
gh repo fork user/repo --clone
git remote add upstream https://github.com/user/repo.git
git fetch upstream && git merge upstream/main    # sync your fork
```

9. Issues & Project Management

- **Issues** track bugs, features, tasks, and discussions — with **labels**, **assignees**, **milestones**, and **issue templates** (`.github/ISSUE_TEMPLATE/`).
- **Closing keywords** in a PR/commit auto-close issues: `Fixes #12` , `Closes #34` , `Resolves #56` .
- **GitHub Projects** — flexible boards/tables (kanban, roadmaps) linking issues & PRs.
- **Discussions** — Q&A / community threads separate from issues.
- **@mentions**, **references** (`#123` , `user/repo#123`), and **task lists** (`- [ ]`) tie everything together.

10. GitHub Actions (CI/CD)

GitHub's built-in automation — run jobs on events (push, PR, schedule, manual).

- **Workflow** = a YAML file in `.github/workflows/`. It has **jobs**, each with **steps**; steps run commands or reusable **actions** (`uses:`).
- **Runners** execute jobs (GitHub-hosted Linux/macOS/Windows, or self-hosted).
- Use **secrets** (`${{ secrets.X }}`) for credentials; cache deps; build matrices to test many versions.

```
# .github/workflows/ci.yml
name: CI
on:
  push: { branches: [main] }
  pull_request:
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 20 }
      - run: npm ci
      - run: npm test
```

Common uses: lint/test on every PR, build & publish images/packages, deploy on merge, scheduled jobs, release automation.

11. Releases, Tags & Packages

- **Tags** mark a point in history; **annotated tags** (`git tag -a v1.0 -m "..."`) are used for versioned releases (semver: `v1.2.3`).
- **Releases** wrap a tag with notes + downloadable **assets** (binaries, archives); auto-generated changelogs available.
- **GitHub Packages / Container Registry (GHCR)** host npm, Maven, NuGet, RubyGems, and **container images** (`ghcr.io/owner/image`).

```
git tag -a v1.0.0 -m "First release" && git push origin v1.0.0
gh release create v1.0.0 ./dist/* --notes "Initial release"
```

12. Authentication & Security

- **Auth for Git:** **HTTPS + Personal Access Token (PAT)** or **SSH keys** (password auth is disabled). The **GitHub CLI** (`gh auth login`) manages credentials for you.
- **2FA** is required for contributors; use **fine-grained PATs** scoped to specific repos/permissions.
- **Security features:** **Dependabot** (dependency updates + alerts), **CodeQL / code scanning**, **secret scanning** (blocks committed credentials), **security advisories**.
- **Branch protection**, **signed commits** (GPG/SSH), and **environments** with required approvals protect important code/deloys.

🔑 Never commit secrets. If you do, **rotate the credential immediately** — rewriting history doesn't un-leak it. Use Actions **secrets** / a secrets manager instead.

13. 📖 GitHub CLI & Useful Commands

```
# GitHub CLI (gh) — auth, repos, PRs, issues, Actions
gh auth login
gh repo clone user/repo
gh pr create / gh pr list / gh pr review / gh pr merge
gh issue create / gh issue list
gh run list / gh run watch          # Actions runs
gh release create v1.0.0

# Handy Git
git clone <url>                    # copy a repo
git remote -v                      # show remotes
git stash / git stash pop          # shelve uncommitted work
git restore --staged file          # unstage
git reset --hard origin/main       # discard local, match remote (destructive)
git revert <sha>                   # undo a commit safely (new commit)
git cherry-pick <sha>              # apply one commit elsewhere
```

14. ✅ Best Practices

- **Small, focused PRs** with clear descriptions — easier to review and revert.
- **Meaningful commits** — conventional style (**feat:** , **fix:** , **docs:**) + a body explaining **why**.
- **Protect main** — require reviews + passing CI; never push directly to it.
- **Branch per feature/fix**; delete branches after merge.
- **Automate with Actions** — lint/test on every PR; gate merges on green checks.
- **Link issues** (**Fixes #N**) and use **CODEOWNERS** + templates for consistency.
- **Never commit secrets**; enable Dependabot, secret scanning, and 2FA.
- Keep a good **README**, **LICENSE**, and **CONTRIBUTING** so others (and future you) can navigate the repo.

15. 🧠 Quick Mental Model

- **Git** = local version control; **GitHub** = hosting + collaboration + automation on top.
- A **repo** holds code + history; you **clone** it, branch, **commit**, and **push**.
- Propose changes via **Pull Requests** → review + CI → **merge** (squash for clean history).
- **Fork & PR** for open source; **shared repo** for teams.
- Track work with **Issues** + **Projects**; auto-close with **Fixes #N**.
- Automate everything with **GitHub Actions** (**.github/workflows/*.yml**).
- Ship with **tags** + **Releases**; distribute via **Packages/GHCR**.

- Secure it: **protect** `main` , **2FA**, **scoped tokens**, **no committed secrets**, **Dependabot + scanning**.
-

 *Notes compiled as a quick reference for GitHub.*